

WHYPOLICE

Software Requirements Specification

AI Search & Streaming Web Application — MVP

Document Version 1.0 | Scope Lock Edition

Project Name	WhyPolice (whypolice.com)
Document Type	Software Requirements Specification (SRS)
Prepared / Developed By	Muhammad Ismaeel
Prepared For	Project Client
Date Issued	August 15, 2026
Agreed Budget	USD \$300
Delivery Deadline	August 31, 2026 (final, confirmed)
Status	Pending Client Sign-Off — Scope Lock

Table of Contents

1. Introduction	3
2. Overall Description	3
3. Reference Product & Design Direction	4
4. Functional Requirements	5
5. External Interface Requirements	7
6. Non-Functional Requirements	8
7. System Architecture & Technology Stack	9
8. Data Model	10
9. UI / UX Design Summary	10
10. Deliverables	11
11. Acceptance Criteria	11
12. Project Timeline, Budget & Milestones	12
13. Assumptions & Constraints	12
14. Approval & Scope Sign-Off	13

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for **WhyPolice**, a gated, subscription-aware, real-time AI search and streaming web application. This document is the single source of truth for the scope of the Minimum Viable Product (MVP) and is intended to be reviewed and approved by the client prior to development lock. Any change to this scope after sign-off will be logged and communicated separately.

1.2 Project Background

WhyPolice was commissioned as an AI search product inspired by the visual and interaction philosophy of `why.com`, an existing AI search application. The client requested an ultra-minimalist, editorial-style interface centered around a search/prompt bar, with mandatory user authentication (to prevent bot/spam abuse), a free/paid subscription model, and real-time streamed answers.

1.3 Scope Summary

WhyPolice requires a visitor to sign up or log in before performing any search. Once authenticated, the user submits a prompt through a centered search bar, and the system streams the response back in real time. Free-tier users receive standard search functionality; Pro-tier users unlock deep search, advanced exports, and downloads. The application also includes session history, a lightweight personal memory feature, and the technical foundation (fields, hooks, UI states) required to connect a payment gateway.

1.4 Definitions, Acronyms & Abbreviations

TERM	DEFINITION
MVP	Minimum Viable Product — the first functional, deliverable release of the product
SSE	Server-Sent Events — a one-way, real-time streaming protocol used to deliver AI responses chunk-by-chunk
JWT	JSON Web Token — the signed token format used by Auth0 to verify a logged-in user's identity on every request
Tier	The user's subscription level: Free or Pro
CRUD	Create, Read, Update, Delete — standard data operations
API	Application Programming Interface — the contract between the frontend and backend
CORS	Cross-Origin Resource Sharing — browser security policy controlling which domains may call the backend

1.5 References

- Reference product: <https://why.com> — used for visual tone, interaction pattern, and product-thinking reference only, not literal design/content copying.
- Client requirement discussion (project inception conversation, August 2026).

2. Overall Description

2.1 Product Perspective

WhyPolice is a new, standalone, full-stack web application. It is not an extension of an existing system. It consists of a Next.js frontend and a FastAPI backend, connected to a managed PostgreSQL database (Neon), a third-party authentication provider (Auth0), and a scaffolded integration point for a payment provider (Stripe).

2.2 Product Functions (High-Level Summary)

- Mandatory account creation / login before any search can be executed.
- Real-time, streamed AI-style search responses via a centered prompt interface.
- Two-tier subscription model (Free / Pro) with visibly different capabilities and an upgrade flow.
- Persistent, revisitable search history per user, with export/clear controls.
- A lightweight, user-controlled "memory notes" feature allowing the system to reference user-stated preferences across searches, with a transparent on-screen indicator whenever a note is used.
- Account management screen showing plan/tier and a billing management entry point.
- Fully responsive, animated, high-polish interface across mobile, tablet, and desktop.

2.3 User Classes and Characteristics

USER CLASS	DESCRIPTION
Guest (unauthenticated)	Can view the public landing page, About, and Pricing pages. Cannot execute a search. Directed to sign up/log in on any search attempt, with the typed query preserved.
Free Tier User	Authenticated user with standard search access, full session history, basic (single-session) export, and full access to the memory notes feature. Deep search and advanced/bulk export are gated and trigger an upgrade prompt.
Pro Tier User	Authenticated user with all Free-tier capabilities plus deep/intensive search and advanced export/download functionality.

2.4 Operating Environment

WhyPolice is a web application accessed via modern desktop and mobile browsers (Chrome, Safari, Edge, Firefox — current and prior major versions). The frontend is deployed on Vercel; the backend runs as a containerized service on Google Cloud Run. The database is a managed, serverless PostgreSQL instance (Neon).

2.5 General Constraints

- Fixed budget of USD \$300 and a fixed delivery deadline of August 31, 2026 — both confirmed and locked.
- Authentication must use Auth0 (client-provided tenant), not a custom-built auth system.
- Database must be Neon (PostgreSQL).
- Streaming must use native Server-Sent Events / `ReadableStream`, not WebSockets or polling.
- Payment gateway integration for this MVP consists of the underlying data fields, API scaffolding, and UI states required to connect Stripe, using test-mode credentials.

2.6 Assumptions and Dependencies

- The client will provide Autho tenant credentials (Domain, Client ID, Client Secret) and Stripe test-mode access in a timely manner, as these are hard dependencies for Sections 4.2 and 4.4 respectively.
- The underlying AI response generation is provided via a template/mock streaming service for this MVP, matching the product's real-time streaming behavior end-to-end.

3. Reference Product & Design Direction

3.1 Design Inspiration

The visual and product-thinking direction is inspired by [why.com](#): an ultra-minimalist, editorial, search-first interface with a warm off-white/cream background, calm and restrained copywriting, and a persisted, revisitable session history pattern. WhyPolice adopts this level of design restraint and product thinking while establishing its own distinct visual identity (see Section 9).

3.2 Feature Alignment with Reference Product

The table below documents the elements of the reference product's experience delivered in this MVP.

REFERENCE FEATURE	STATUS	NOTES
Minimal, editorial, cream-background UI	Included	Core visual direction for the entire application
Centered, search-first landing experience	Included	Primary interaction pattern of the product
Real-time streamed responses	Included	Delivered via Server-Sent Events
Session history ("paths"): list, export, import, clear	Included	Export/clear fully functional; import may be presented as a near-term placeholder
Calm "search interrupted / try again" recovery state	Included	Applied to any streaming interruption
Mandatory login before use	Included	Client-specific requirement, added on top of the reference product's own behavior
Subscription tiers & paywall	Included	Client-specific requirement, added on top of the reference product's own behavior
Lightweight personal memory notes + on-screen reference indicator	Included — Lightweight Version	User-authored notes only; see Section 4.6 for exact boundaries

4. Functional Requirements

4.1 Public Access & Marketing Pages

- **FR-1.1** — The system shall provide a public landing page featuring a headline, a centered search/prompt bar, and example prompt suggestions.
- **FR-1.2** — The system shall provide public About and Pricing pages, accessible without authentication.
- **FR-1.3** — The system shall provide public Privacy Policy and Terms of Service pages.
- **FR-1.4** — The system shall provide search-engine metadata (titles, descriptions, sitemap, robots directives) for all public pages.

4.2 Authentication & Access Control

- **FR-2.1** — The system shall require every user to be authenticated before executing any search query.
- **FR-2.2** — The system shall provide dedicated, on-brand Login and Sign-Up pages, which hand off to a branded Autho authentication flow.
- **FR-2.3** — If an unauthenticated visitor attempts to submit a search, the system shall preserve the typed query and route the user to sign up, auto-filling the query upon successful registration.
- **FR-2.4** — The system shall create a corresponding user record (default tier: Free) automatically upon a user's first authenticated request.
- **FR-2.5** — The backend shall independently verify the authenticity of every request to protected endpoints via signed token verification, regardless of frontend-level access control.
- **FR-2.6** — The system shall provide a functioning logout capability, clearing the user's session.

4.3 Search & Real-Time Streaming

- **FR-3.1** — The system shall provide a centered search input where an authenticated user can submit a natural-language prompt.
- **FR-3.2** — The system shall stream the generated response back to the user progressively (chunk-by-chunk), not as a single delayed block.
- **FR-3.3** — The system shall visually indicate the following distinct states: idle, typing, submitting, streaming, complete, interrupted/error, and rate-limited.
- **FR-3.4** — If a stream is interrupted, the system shall present a calm recovery message with a retry action.
- **FR-3.5** — The system shall apply a per-user rate limit to search submissions and shall return a clear, on-brand message (including a retry-after indication) when the limit is exceeded.
- **FR-3.6** — The system shall offer a "deep search" option, restricted to Pro-tier users; an attempt by a Free-tier user shall trigger an upgrade prompt rather than executing the request.

- **FR-3.7** — Every completed search shall be persisted as a session, associated with the requesting user.

4.4 Subscription Tiers & Paywall

- **FR-4.1** — The system shall maintain a subscription tier (Free or Pro) for every user account.
- **FR-4.2** — The system shall visibly differentiate Free and Pro capabilities throughout the interface (badges, gated actions, plan comparison).
- **FR-4.3** — Any attempt by a Free-tier user to access a Pro-restricted capability shall present an upgrade prompt rather than fail silently or behave inconsistently.
- **FR-4.4** — The system shall provide a pricing comparison page (public) and a contextual upgrade page (authenticated).
- **FR-4.5** — The system shall include the underlying data fields, API scaffolding, and UI states necessary to connect a live payment gateway (Stripe) in a subsequent phase, using test-mode credentials for this MVP.

4.5 Search History

- **FR-5.1** — The system shall provide a history view listing all of a user's past search sessions, most recent first.
- **FR-5.2** — The system shall allow a user to open any past session and view its full content in a read-only replay view.
- **FR-5.3** — The system shall allow a user to export a single session as a downloadable file, available to all tiers at no restriction.
- **FR-5.4** — The system shall allow a user to clear their entire search history, with a confirmation step to prevent accidental loss.
- **FR-5.5** — An advanced/bulk export or formatted export (e.g. document format) capability, if offered, shall be restricted to Pro-tier users.

4.6 Lightweight Memory & Transparency

A personal, user-controlled memory feature: the user directly authors what the system remembers about them, and the system is transparent about when that memory is used.

- **FR-6.1** — The system shall allow an authenticated user to create, edit, and delete short, self-authored "memory notes" describing their own preferences or context.
- **FR-6.2** — Memory notes shall be private to the account that created them and shall never be shared across users.
- **FR-6.3** — When a search response makes use of one or more memory notes, the system shall display a brief, visible indicator (e.g. "Memory referenced: ...") identifying which note(s) were used.
- **FR-6.4** — If no memory notes were used for a given search, no indicator shall be shown (absence of the indicator, not an empty/placeholder state).

- **FR-6.5** — Memory notes and their referencing shall be available identically to Free and Pro tier users, with no tier restriction.

4.7 Account Management

- **FR-7.1** — The system shall provide an account screen displaying the user's profile information and current subscription tier.
- **FR-7.2** — The system shall provide an entry point to manage billing (scaffolded for MVP, connecting to a live billing portal in a subsequent phase).

5. External Interface Requirements

5.1 User Interfaces

All user-facing screens shall conform to the design system defined in the project's Theme Guideline documentation, covering color, typography, spacing, componentry, and motion. A summary is provided in Section 9 of this document.

5.2 API Interfaces (Frontend ↔ Backend)

METHOD	ENDPOINT	PURPOSE
POST	/api/search/stream	Submit a search prompt and receive a streamed response (authenticated)
GET	/api/search/history	Retrieve the current user's list of past sessions
GET	/api/search/{session_id}	Retrieve one session's full content
DELETE	/api/search/history	Clear the current user's search history
GET	/api/me	Retrieve the current user's profile and subscription tier
GET / POST / PATCH / DELETE	/api/memory	Manage the current user's memory notes
POST	/api/billing/*	Payment gateway scaffolding (checkout session, webhook receiver)
GET	/api/health	Service health check

5.3 Software Interfaces (Third-Party Services)

SERVICE	ROLE
Auth0	User authentication, session management, identity verification
Neon	Managed, serverless PostgreSQL database
Stripe	Payment processing (test-mode scaffolding for MVP)
Vercel	Frontend hosting and deployment
Google Cloud Run	Backend service hosting (Docker-based, serverless)

6. Non-Functional Requirements

PERFORMANCE

NFR-1 — Search responses shall begin streaming visibly to the user within a short, perceptible delay of submission, without a blocking full-page wait. **NFR-2** — Public marketing pages shall achieve strong performance scores under standard web performance auditing (Lighthouse Performance ≥ 85).

SECURITY

NFR-3 — All protected API endpoints shall verify authentication tokens server-side on every request. **NFR-4** — No credentials, API keys, or database connection strings shall be committed to source control at any point. **NFR-5** — Cross-origin access to the backend shall be restricted to explicitly approved origins. **NFR-6** — All error responses shall follow a consistent, non-revealing format; internal error details (e.g. stack traces) shall never be exposed to the client.

USABILITY & ACCESSIBILITY

NFR-7 — The interface shall be fully operable via keyboard navigation, with visible focus states on all interactive elements. **NFR-8** — Text and background color combinations shall meet WCAG AA contrast standards. **NFR-9** — The interface shall respect the user's reduced-motion accessibility preference.

RESPONSIVENESS

NFR-10 — The application shall be fully functional and visually polished across mobile (375px and up), tablet, and desktop viewport widths, with no loss of functionality or broken layout at any supported size.

RELIABILITY & CONSISTENCY

NFR-11 — Every screen that loads remote data shall present a loading indicator matching the shape of its final content, avoiding blank or jarring transitions. **NFR-12** — Navigation between screens shall be smooth and continuous, without full blank-page reloads.

MAINTAINABILITY & PORTABILITY

NFR-13 — The complete application (frontend and backend) shall be containerized and runnable locally via a single command. **NFR-14** — The codebase shall be delivered as clean, documented source code via version control transfer, free of proprietary lock-in.

7. System Architecture & Technology Stack

7.1 Architecture Overview

WhyPolice follows a decoupled two-service architecture: a Next.js frontend responsible for all user interface and interaction, and a FastAPI backend responsible for authentication verification, business logic, subscription enforcement, and communication with the database. The frontend never accesses the database directly — all data operations pass through the backend API, which serves as the system's single source of truth.

7.2 Technology Stack

LAYER	TECHNOLOGY
Frontend Framework	Next.js (React, App Router)
UI Component Library	shadcn/ui (Tailwind CSS + Radix UI primitives)
Animation	Framer Motion, GSAP, animate.css
Backend Framework	FastAPI (Python, fully asynchronous)
Real-Time Communication	Server-Sent Events (native browser streaming, no external library)
Authentication	Auth0
Database	Neon (serverless PostgreSQL)
Payments	Stripe (test-mode scaffolding)
Containerization	Docker (multi-stage builds), Docker Compose
Hosting	Vercel (frontend), Google Cloud Run (backend)

7.3 Repository Structure

The project is delivered as a single repository containing two clearly separated top-level directories: `/frontend` and `/backend`, each independently deployable, each with its own dependency management and environment configuration.

8. Data Model

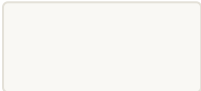
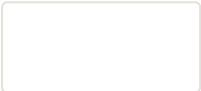
ENTITY	KEY FIELDS
users	identity reference, email, subscription tier, account creation date, billing reference (optional)
search_sessions	owning user, session title, creation date
search_messages	parent session, role (user/assistant), content, deep-search flag, referenced memory notes, creation date
memory_notes	owning user, note content, created/updated timestamps

9. UI / UX Design Summary

9.1 Visual Direction

An editorial-minimal aesthetic: a warm off-white/cream background (never pure white), confident near-black typography (never pure black), a single restrained accent color, and generous whitespace — matching the reference product's restraint while establishing a distinct visual identity via a warm terracotta accent and an editorial serif display typeface for headlines.

9.2 Core Color Palette

					
Background #F9F8F4	Elevated Surface #FFFFFF	Primary Text #1A1A18	Accent #C9622B	Success #2E7D4F	Danger #B23A2E

9.3 Typography

A clean sans-serif typeface is used for all interface text and body copy; a warm editorial serif is reserved exclusively for the hero headline and major section headlines, giving the product a distinct, premium identity. A monospace typeface is used sparingly for meta information.

9.4 Motion & Interaction

Motion is treated as a first-class product quality signal: consistent timing and easing standards, meaningful (not decorative) animation, full support for reduced-motion accessibility preferences, and dedicated attention to eliminating visual flicker, layout shift, or inconsistent state — with loading skeletons matching final content shape throughout the application rather than generic spinners or blank screens.

9.5 Responsiveness

The interface is designed mobile-first and verified at defined breakpoints from 375px through large desktop widths, with particular attention paid to the search and streaming experience remaining fully usable on small screens.

10. Deliverables

- Complete, working source code for both frontend and backend, delivered via the project's version-controlled repository.
- A working Docker configuration allowing the entire application to be run locally via a single command.
- A clear setup document (README) describing environment configuration and local run instructions.
- A live, deployed preview of the application (frontend and backend), connected end-to-end.
- This Software Requirements Specification document, serving as the scope-of-record for the engagement.

11. Acceptance Criteria

The MVP shall be considered complete and accepted when all of the following criteria are demonstrably true:

1. A guest attempting to search is redirected/prompted to authenticate; no search executes without a logged-in session.
2. A new user can successfully sign up and log in end-to-end via Auth0.
3. A logged-in user can submit a prompt and observe the response streaming in visibly, in real time, matching the intended minimal aesthetic.
4. Free and Pro tier states are visibly distinct in the interface, and any gated action presents an upgrade prompt to Free-tier users.
5. The full application runs locally with a single Docker command; the setup document explains all required configuration.
6. The application is fully usable and visually polished on mobile, tablet, and desktop viewport sizes.
7. No credentials or secrets exist anywhere in the delivered source code.
8. A live, working deployed preview exists and functions end-to-end.
9. A user can create, edit, and delete memory notes, and any search that references one displays the transparency indicator correctly.

12. Project Timeline, Budget & Milestones

ITEM	DETAIL
Agreed Budget	USD \$300
Delivery Deadline	August 31, 2026 (confirmed final)
Development Approach	Sequential, milestone-based build: environment & foundation setup → design system → public pages → authentication → data layer → core search & streaming → history → memory & transparency → subscription/paywall → responsiveness & polish pass → containerization → deployment → final verification passes
Verification	Each milestone is functionally tested and adversarially reviewed for defects before being considered complete; a full end-to-end walkthrough and scope-compliance audit are performed prior to final delivery

13. Assumptions & Constraints

- The client will provide Autho tenant access/credentials and Stripe account access in a timely manner, as both are hard dependencies for the corresponding build phases.
- The client will confirm the domain registrar details for whypolice.com prior to the deployment/go-live phase.
- The underlying AI response generation uses a template-based streaming mechanism for this MVP, delivering the full real-time streaming experience end-to-end.

14. Approval & Scope Sign-Off

By signing below, the client confirms that this Software Requirements Specification accurately reflects the agreed scope of work for the WhyPolice MVP, and that development will proceed on this basis. Any requested changes following sign-off will be documented as a separate scope amendment.

Client Signature Date: _____

Muhammad Ismaeel (Developer) Date: _____

WhyPolice — Software Requirements Specification • Version 1.0 • Developed by Muhammad Ismaeel • August 2026